

TECHNICAL HIRING ASSESSMENT

Backend Development Assessment – Python

Project: Basic Banking Application

Target Audience	Final-year B.Tech students / entry-level software engineering candidates
Role Track	Backend Development
Core Technologies	Python, FastAPI, REST API, JWT Authentication, SQL database, SQLAlchemy (or equivalent ORM), Pydantic
Database	SQLite, PostgreSQL or MySQL
Submission	Public GitHub / GitLab repository link

Please read this document completely before you begin. It contains all the information required to complete the assessment.

Contents

1. Introduction
2. API Summary
3. User Registration
4. User Login
5. Get Account Details
6. Deposit Money
7. Withdraw Money
8. Transfer Money
9. Transaction History
10. Logout
11. Database
12. Authentication
13. Validation
14. Error Handling
15. Security Requirements
16. Project Structure
17. API Documentation
18. Testing
19. Edge Cases
20. README
21. Optional / Bonus Features
22. Expected Submission

1. Introduction

In this assessment you will build a **Basic Banking Application**: a REST API that allows users to register, authenticate, manage their bank account, and perform basic banking transactions.

The following areas will be considered:

- Python fundamentals
- FastAPI
- REST API design
- Database design
- Authentication
- Authorization
- Input validation
- Error handling
- Transaction handling
- Clean code and project structure

1.1 Technology Stack

Technology	Usage	Level
Python	Programming language	Mandatory
FastAPI	Web framework for the REST API	Mandatory
JWT	Authentication for protected endpoints	Mandatory
SQL database	SQLite, PostgreSQL or MySQL (your choice)	Mandatory
SQLAlchemy or equivalent ORM	Database access layer	Mandatory
Pydantic	Request / response models and validation	Mandatory

1.2 How Requirements Are Marked

Label	Meaning
Mandatory	Must be implemented for the submission to be considered complete.
Optional	Bonus features. Not required to complete the assignment. Attempt only after all mandatory requirements are done.

2. API Summary

MANDATORY

Your application must expose the following endpoints. Each endpoint is described in detail in the sections that follow.

#	Method	Endpoint	Auth	Description
1	POST	/api/auth/register	No	Register a new user
2	POST	/api/auth/login	No	Log in and receive a JWT access token
3	GET	/api/accounts/me	Yes	Get the current user's account details
4	POST	/api/accounts/deposit	Yes	Deposit money
5	POST	/api/accounts/withdraw	Yes	Withdraw money
6	POST	/api/accounts/transfer	Yes	Transfer money to another account
7	GET	/api/accounts/transactions	Yes	Get the current user's transaction history
8	POST	/api/auth/logout	Yes	Log out

3. User Registration

MANDATORY

PUBLIC

POST /api/auth/register

REQUEST BODY

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "Password123"
}
```

Requirements

- Name is required
- Email must be valid
- Email must be unique
- Password must be securely hashed
- Password must **never** be stored as plain text
- Create a user account

4. User Login

MANDATORY

POST /api/auth/login

PUBLIC

REQUEST BODY

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

The API should authenticate the user and return a JWT access token.

EXAMPLE RESPONSE

```
{
  "accessToken": "JWT_TOKEN",
  "tokenType": "bearer"
}
```

5. Get Account Details

MANDATORY

GET

/api/accounts/me

AUTH REQUIRED

Return information about the currently authenticated user's bank account. Example information:

Field	Description
Account ID	Unique identifier of the account
Account number	Bank account number
Account holder name	Name of the account owner
Current balance	Available balance
Account creation date	Date and time the account was created

ACCESS RULE

A user must only be able to access **their own** account.

6. Deposit Money

MANDATORY

POST

/api/accounts/deposit

AUTH REQUIRED

REQUEST BODY

```
{  
  "amount": 5000  
}
```

Requirements

- Amount must be greater than zero
- Increase the user's account balance
- Record the transaction

7. Withdraw Money

MANDATORY

AUTH REQUIRED

POST

`/api/accounts/withdraw`

REQUEST BODY

```
{  
  "amount": 2000  
}
```

Requirements

- Amount must be greater than zero
- User cannot withdraw more than the available balance
- Deduct the amount from the account
- Record the transaction

8. Transfer Money

MANDATORY

AUTH REQUIRED

POST

`/api/accounts/transfer`

REQUEST BODY

```
{  
  "receiverAccountId": 102,  
  "amount": 1500  
}
```

Requirements

- Sender must be authenticated
- Receiver account must exist
- Amount must be greater than zero
- Sender must have sufficient balance

- Sender cannot transfer money to their own account
- Deduct money from the sender
- Add money to the receiver
- Record the transaction

IMPORTANT — ATOMICITY

The transfer operation **must be atomic**.
If any part of the transfer fails, the complete operation must be rolled back.

9. Transaction History**MANDATORY****GET****/api/accounts/transactions****AUTH REQUIRED**

Return the authenticated user's transaction history.

EXAMPLE RESPONSE

```
[
  {
    "id": 1,
    "type": "DEPOSIT",
    "amount": 5000,
    "status": "SUCCESS",
    "createdAt": "2026-09-25T10:30:00"
  },
  {
    "id": 2,
    "type": "WITHDRAW",
    "amount": 1000,
    "status": "SUCCESS",
    "createdAt": "2026-09-25T11:00:00"
  }
]
```

ACCESS RULE

The user must **not** be able to view another user's transactions.

10. Logout**MANDATORY****POST****/api/auth/logout****AUTH REQUIRED**

Implement an appropriate logout mechanism. Since JWT authentication is normally stateless, you must **explain your chosen logout strategy in the README**. You may use any of the following:

- Token blacklist

- Refresh token mechanism
- Short-lived access tokens
- Server-side token / session tracking

11. Database

MANDATORY

Use a relational database. Acceptable options: SQLite, PostgreSQL or MySQL.

11.1 Suggested Entities

USER

Column	Column	Column
id	name	email
password_hash	created_at	

ACCOUNT

Column	Column	Column
id	user_id	account_number
balance	created_at	

TRANSACTION

Column	Column	Column
id	account_id	type
amount	status	reference_id
created_at		

You may modify the database structure if required. Explain your design decisions in the README.

12. Authentication

MANDATORY

Protected APIs must require JWT authentication, passed in the request header:

REQUEST HEADER

Authorization: Bearer <JWT_TOKEN>

Requests without valid authentication must return an appropriate HTTP status code. The API must handle:

- Missing token

- Invalid token
- Expired token
- Unauthorized access

13. Validation

MANDATORY

The API must validate input data using appropriate Pydantic models and validation. Examples of invalid input to handle:

Input Case	Input Case
Invalid email	Empty name
Weak / invalid password	Negative amount
Zero amount	Invalid account ID
Non-existent receiver	Insufficient balance

14. Error Handling

MANDATORY

Use appropriate HTTP status codes and meaningful error responses. Examples of status codes:

Status Code	Name
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error

Avoid exposing sensitive implementation details in API error responses.

15. Security Requirements

MANDATORY

All of the following are mandatory:

#	Requirement
1	Passwords must be hashed

#	Requirement
2	Passwords must never be returned through APIs
3	JWT authentication must be implemented
4	Users must only access their own account information
5	Users must only access their own transactions
6	Input must be validated
7	Sensitive information must not be exposed in error responses

16. Project Structure

MANDATORY

Use a clean and maintainable project structure. For example:

```
EXAMPLE STRUCTURE
app/
  main.py
  api/
  models/
  schemas/
  services/
  repositories/
  core/
  db/
  tests/
  requirements.txt
  README.md
```

The exact structure is left to you.

17. API Documentation

MANDATORY

FastAPI's Swagger / OpenAPI documentation must be available. Ensure that all endpoints, including protected ones, can be tested through the generated API documentation.

18. Testing

MANDATORY

Write tests for important functionality. At a minimum, test the following:

Test Area	Test Area
Registration	Login
Deposit	Withdrawal

Test Area	
Transfer	
Insufficient balance	
Test Area	
Unauthorized request	

Backend Development Assessment - Python

19. Edge Cases

MANDATORY

Your implementation must consider the following edge cases:

#	Edge Case	#	Edge Case
1	Duplicate email	8	Invalid JWT
2	Invalid login credentials	9	Expired JWT
3	Negative deposit	10	Non-existent account
4	Zero deposit	11	Non-existent receiver
5	Negative withdrawal	12	Transfer to self
6	Zero withdrawal	13	Database failure during transfer
7	Insufficient balance	14	Concurrent transactions

20. README

MANDATORY

Include a README.md at the root of the repository containing the following sections:

#	Section	#	Section
1	Project overview	8	API documentation
2	Technology stack	9	Authentication approach (including logout strategy)
3	Prerequisites	10	Database design
4	Installation instructions	11	Design decisions
5	Environment variables	12	How to run tests
6	Database setup	13	Known limitations
7	How to run the application		

21. Optional / Bonus Features

OPTIONAL / BONUS

The following features are **optional** and are **not required** to complete the core assignment. Complete all mandatory requirements first.

Feature	Feature	Feature
Refresh tokens	Pagination for transaction history	Transaction filtering
Transaction sorting	Account number generation	Admin API
API rate limiting	Docker support	Additional unit tests
Integration tests	Database migrations	Logging
Redis caching		

If you implement any bonus features, list them in your README.

22. Expected Submission

MANDATORY

Submit the complete source code through a **GitHub** or **GitLab** repository and share the repository link.

22.1 The repository must contain

- Source code
- Tests (tests/)
- requirements.txt (or equivalent dependency file)
- README.md
- A sample environment file (e.g. .env.example) listing required environment variables without real secret values
- Any configuration files required to run the project

22.2 The repository must not contain

- Virtual environment folders (e.g. venv/, .venv/)
- Real secrets, keys or passwords (e.g. a populated .env file)
- __pycache__/ and other generated files

BEFORE YOU SUBMIT

Clone your repository into a fresh folder, set it up using only the steps in your README, start the server, open the Swagger documentation, and run the tests. Make sure everything works. Ensure the repository is accessible to the reviewers.